Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №7

По дисциплине «Логика и основы алгоритмизации в инженерных задачах»

На тему «Обход графа в глубину»

Выполнили студенты группы 25ВВВ3:
Сергеева С.М.
Иванова С.А.
Приняли:
к.т.н., доцент Юрова О. В.
к.т.н., доцент Деев М. В.

Пенза 2026

Цель: изучить алгоритм обхода неориентированного графа в глубину, научиться представлять граф матрицей и списками смежности, реализовать рекурсивный обход для обеих форм представления и выполнить переход от рекурсивной реализации к нерекурсивной для матрицы смежности.

Ход работы.

В начале исходного файла подключены стандартные библиотеки.

Для представления списка смежности в программе объявлена структура Node. В каждом её элементе поле vertex хранит номер соседней вершины, а next указывает на следующий элемент списка. Если next равен NULL, значит, текущий элемент последний. Для каждой вершины графа создаётся отдельный связанный список, поэтому дополнительно используется массив указателей Node** list.

Основное представление графа — квадратная матрица смежности A размером n × n. В ней значение 1 означает, что между двумя вершинами есть ребро, а 0 — что ребра нет. Поскольку граф неориентированный, матрица симметрична: если A[i][j] равно 1, то A[j][i] также равно 1. Главная диагональ остаётся нулевой, так как при генерации петли не добавляются.

Матрица создаётся функцией createMatrix. Сначала malloc выделяет память под массив указателей на строки, а затем calloc выделяет каждую строку и сразу заполняет её нулями. После каждого выделения проверяется результат. Если память для очередной строки получить не удалось, функция освобождает уже созданные строки и массив указателей, после чего возвращает NULL. Так программа не оставляет частично выделенную матрицу.

Заполнение выполняет randomFill. Вложенные циклы перебирают только пары вершин, для которых j больше i. Для каждой пары рассчитывается случайное число r как rand() / RAND_MAX. Если r меньше введённой вероятности p, в матрице одновременно устанавливаются A[i][j] = 1 и A[j][i] = 1. Ввод p ограничен диапазоном от 0 до 1: при p = 0 рёбра не появляются, а при p = 1 соединяются все различные пары вершин. Функция printMatrix затем выводит получившуюся матрицу с обозначениями x1, x2 и т.д.

Для освобождения матрицы предусмотрена отдельная функция freeMatrix. Она сначала освобождает каждую строку, а затем сам массив указателей. Эта

функция вызывается при создании нового графа, чтобы удалить предыдущую матрицу, и в конце main перед завершением программы.

Для третьего пункта задания матрица преобразуется в списки смежности. Функция createAdjacencyList выделяет массив указателей Node* и для каждой строки матрицы создаёт узлы только для тех столбцов, где стоит единица. Поскольку столбцы просматриваются слева направо, соседи попадают в список в порядке возрастания номеров. Матрица и список задают один и тот же порядок просмотра соседей.

В функции printAdjacencyList списки выводятся на экран в виде строк «x1: x2 x3 ...». Если у вершины нет соседей, выводится соответствующее сообщение. После выполнения выбранной операции функция freeAdjacencyList проходит по каждому списку, освобождает его узлы и в конце освобождает массив указателей. Память под временное представление графа не остаётся занятой после возврата в меню.

Рекурсивный обход по матрице выполняется функцией dfsMatrix. При входе в функцию текущая вершина сразу отмечается в visited и печатается. Затем цикл проверяет все элементы соответствующей строки матрицы. Если найдено ребро к вершине, которая ещё не посещалась, для неё вызывается dfsMatrix. Когда рекурсивный вызов завершается, выполнение продолжается с того места, где был найден сосед. Именно так программа возвращается назад по ранее пройденному пути.

Массив visited выделяется в пункте меню перед началом обхода и изначально заполняется нулями с помощью calloc. Ноль означает, что вершина ещё не посещена, единица — что она уже была обработана. Внешний цикл запускает dfsMatrix для каждой вершины, которая всё ещё имеет отметку 0. Если из одной компоненты нельзя добраться до остальных вершин, обход начинается заново из следующей непосещённой вершины.

Вариант для списков смежности устроен почти так же. Функция dfsListRec отмечает и выводит текущую вершину, после чего перебирает узлы её списка. Если сосед ещё не посещён, вызывается dfsListRec для этого соседа. Функция dfsList выделяет массив visited и запускает рекурсивную процедуру для всех компонент графа. Отличие этого варианта в том, что для поиска соседей просматривается связанный список, а не вся строка матрицы.

Четвёртый пункт выполняется функцией dfsIterative. В ней рекурсивные вызовы заменены собственным стеком, который хранится в двух динамических массивах. stackVertex содержит вершины текущего пути, а stackNext хранит для каждой из них номер столбца, с которого нужно продолжить поиск следующего соседа. Переменная top указывает на верхний занятый элемент стека.

Когда находится непосещённый сосед j, программа сначала сохраняет в stackNext[top] значение j + 1. Это означает, что после возвращения к текущей вершине просмотр матрицы продолжится со следующего столбца. Затем найденная вершина отмечается как посещённая, выводится и помещается на новый уровень стека. Если непосещённых соседей больше нет, top уменьшается на единицу — программа возвращается к предыдущей вершине. Таким образом, два массива вместе хранят ту информацию, которую при рекурсивном варианте система сохраняет в стеке вызовов функций. Поэтому при одинаковом порядке просмотра соседей нерекурсивный вариант выводит те же вершины в том же порядке.

В функции main организован цикл главного меню и обработка выбора через switch.

```
Главное меню:
1 - Создать граф и вывести матрицу смежности
2 - Рекурсивный обход в глубину по матрице
3 - Рекурсивный обход в глубину по спискам смежности (*)
4 - Нерекурсивный обход в глубину по матрице (*)
0 - Выход
Ваш выбор: 1
Введите количество вершин: 6
Введите вероятность появления ребра (от 0 до 1): 0,7

Матрица смежности графа G:
    x1  x2  x3  x4  x5  x6
x1  0   1   1   1   1   1
x2  1   0   0   0   0   1
x3  1   0   0   1   1   1
x4  1   0   1   0   1   1
x5  1   0   1   1   0   1
x6  1   1   1   1   1   0
```

Рисунок 1 – Создание графа и вывод матрицы смежности (пункт меню 1)

Граф сохраняется в памяти, поэтому для последующих обходов не требуется генерировать его заново.

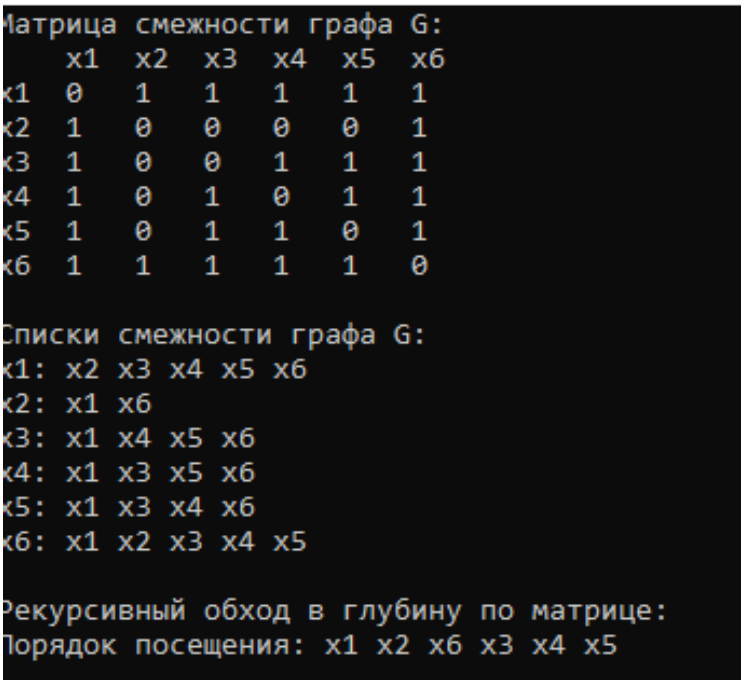


Рисунок 2 – Рекурсивный обход в глубину по матрице смежности (пункт меню 2)

Во втором пункте экран очищается, после чего программа повторно показывает матрицу и соответствующие ей списки смежности. Затем запускается dfsMatrix. Для приведённого графа порядок посещения получился таким: x1, x2, x4, x5, x3. Сначала обход идёт от x1 к x2, далее к x4 и x5; после возврата обнаруживается ещё не посещённая вершина x3.

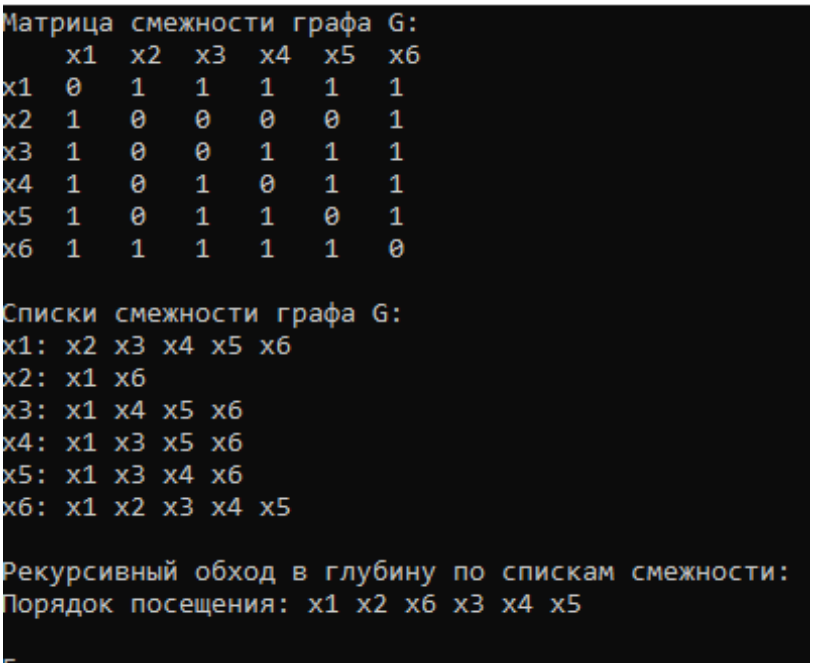


Рисунок 3 – Рекурсивный обход в глубину по спискам смежности (пункт меню 3)

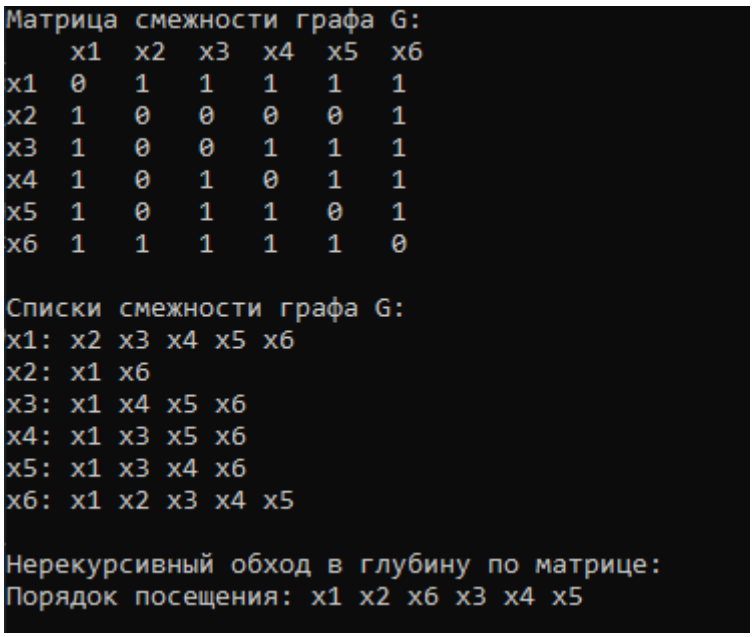


Рисунок 4 – Нерекурсивный обход в глубину по матрице смежности (пункт меню 4)

В четвёртом пункте тот же обход выполняется без рекурсивных вызовов. Возврат к предыдущей вершине осуществляется уменьшением top, а продолжение просмотра её соседей — по сохранённому значению stackNext.

Вывод.

В ходе лабораторной работы мы реализовали генерацию неориентированного графа и его представление матрицей смежности. Также был построен вариант хранения графа в виде списков смежности. Для обоих представлений выполнен рекурсивный обход в глубину, а для матрицы дополнительно реализован нерекурсивный вариант с собственным стеком. Во время работы мы использовали динамическое выделение и освобождение памяти, массив отметок посещения и меню для выбора операций. На одном и том же графе все три алгоритма дали одинаковый порядок посещения вершин, что подтвердило корректность нерекурсивного преобразования.